

**INSTITUTO POLITÉCNICO
DO CÁVADO E DO AVE
ESCOLA SUPERIOR
DE TECNOLOGIA**

Programação Orientada a Objetos
Relatório do Trabalho Prático Fase 2

Tiago Ferreira 27980

Data
Dezembro de 2024

Ano
2024 - 2025

Curso
Engenharia de Sistemas Informáticos

Conteúdo

1	Introdução	4
2	Estrutura do Projeto (Arquitetura N-Tier)	5
3	Funcionalidades Implementadas	6
3.1	Gestão de Reservas	6
3.2	Gestão de Apartamentos	7
3.3	Gestão de Utilizadores	8
4	Aspetos Adicionais do Projeto	9
4.1	Regras de Negócio	9
4.2	Exceções e Tratamento de Erros	10
4.3	Validações de Dados	11
4.4	Interação entre Camadas	13
5	Dificuldades e Soluções	14
5.1	Guardar Listas Em Ficheiros Binários	14
5.2	Ler Listas Em Ficheiros Binários	14
6	Melhorias Futuras	15
6.1	Persistência de Dados	15
6.2	Interface Gráfica	15
6.3	Integração com uma Base de Dados	15
7	Diagrama de Classes	16
8	Conclusão	18

Lista de Códigos

1	Cancelar uma reserva	6
2	Atualizar apartamento	7
3	Remover um utilizador	8
4	Criar um Utilizador	9
5	Exceções de Utilizador	10
6	Validação Geral	11
7	Validações Específicas	11
8	Guardar Ficheiro Binario	14
9	Ler Ficheiro Binario	14

Lista de Figuras

1	Estrutura de camadas(bibliotecas) do projeto.	5
2	Camadas usadas na camada Dados	13
3	Diagrama de Classes do Sistema (Exceções)	16
4	Diagrama de Classes do Sistema (Objetos e Dados)	16
5	Diagrama de Classes do Sistema (Regras de Negocio, Main e Validações)	17

1 Introdução

O presente trabalho tem como objetivo o desenvolvimento de um projeto de gestão de alojamentos turísticos, utilizando a linguagem de programação C#. Este projeto foi feito de forma a administrar utilizadores, alojamentos(apartamentos) e as suas respetivas reservas.

O projeto segue uma abordagem baseada na cadeira de Programação Orientada a Objetos (POO). O código foi desenvolvido de maneira a ser adaptável a melhorias, facilitando trabalhos futuros, se for necessário.

Nesta segunda fase do trabalho, foi dada continuidade ao trabalho iniciado na Fase 1. As melhorias envolveram o aprimoramento da estrutura do código para uma arquitetura N-Tier, regras de negócio, introdução de classes para validação e, por fim, foi adicionado exceções.

Entre as principais funcionalidades, estão:

- Adicionar, listar, atualizar e remover clientes.
- Gestão de reservas, considerando datas, número de hóspedes.
- Gestão de Apartamentos.

Por fim, o projeto foi desenvolvido com o objetivo de simular a operação de um sistema de alojamentos turísticos.

2 Estrutura do Projeto (Arquitetura N-Tier)

A estrutura do projeto foi reorganizada com base na arquitetura N-Tier para a Fase 2, de forma a promover a separação de responsabilidades e facilitar a manutenção do código. O trabalho é dividido em várias camadas, cada uma com sua responsabilidade, conforme está abaixo:

- **Dados:** Contém as classes responsáveis pela manipulação das listas dos dados, como ‘Apartamentos’, ‘Reservas’, e ‘Utilizadores’.
- **Excecoes:** Define as classes de exceção para tratar erros específicos, como ‘ExcecaoApartamentoRegras’ e ‘ExcecaoReservaRegras’.
- **Main:** Inclui o ponto de entrada do programa (*Program.cs*).
- **ObjetosDeNegocio:** Contém classes de modelo, como ‘Apartamento’, ‘Reserva’ e ‘Utilizador’.
- **RegrasDeNegocio:** Contém as regras específicas para cada funcionalidade, como ‘RegrasApartamentos’ e ‘RegrasReservas’. Por exemplo, apenas um administrador pode remover uma reserva, mas cancelar uma reserva, o moderador já consegue.
- **Validacoes:** Inclui validações de atributos para utilizadores, reservas e apartamentos como ‘ValidarApartamentos’ e ‘ValidarReservas’.

Separar em camadas facilita a manutenção no trabalho, pois assim cada camada pode ser modificada sem afetar diretamente as outras. Para além disto a arquitetura N-Tier ajuda também a isolar os dados, proporcionando um projeto mais seguro.

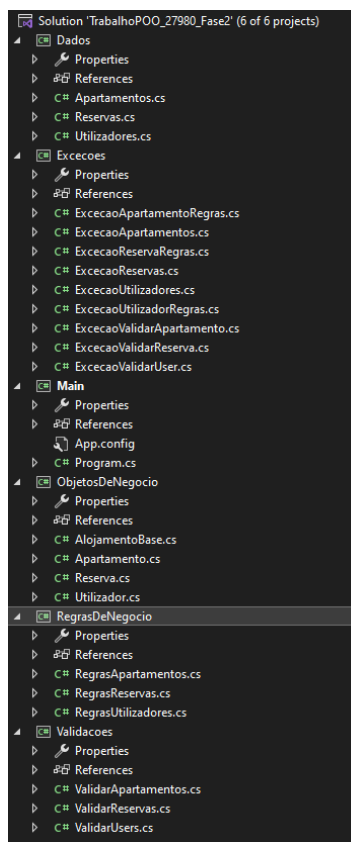


Figura 1: Estrutura de camadas(bibliotecas) do projeto.

3 Funcionalidades Implementadas

Esta seção do relatório descreve as funcionalidades principais implementadas no projeto de gestão de alojamentos, abordando como a gestão de apartamentos, reservas e utilizadores é feita.

3.1 Gestão de Reservas

As principais funcionalidades na gestão de reservas são:

- **Criação de reservas:** Cria uma reserva com os seguintes atributos: ID da reserva, apartamento, utilizador, data de início, data final e total de hóspedes.
- **Atualização de reservas:** Atualiza os atributos da reserva, caso seja necessário.
- **Cancelamento de reservas:** Cancela a reserva, mudando seu id para -100 e coloca o apartamento como disponível novamente.
- **Remoção de reservas:** Remove da lista de reservas e cancela a reserva.

Exemplo de código para o cancelamento de uma reserva:

```
1 public static bool CancelarReserva(int id, int idApartamento)
2 {
3     Reserva r = BuscarReservaPorId(id);
4     Apartamento ap = Apartamentos.BuscarApartamentoPorId(idApartamento);
5
6     //Verificar se o apartamento existe
7     if (ap == null)
8     {
9         throw new ExcecaoApartamento(2);
10    }
11
12    //Verificar se a reserva existe
13    if (r != null)
14    {
15        r.Status = false;
16        r.Id = -100;
17        //Atualiza a disponibilidade do apartamento para disponível.
18        Apartamentos.AtualizarStatusApartamento(ap);
19
20        return true;
21    }
22    throw new ExcecaoReservas(2);
23 }
```

Código 1: Cancelar uma reserva

3.2 Gestão de Apartamentos

As principais funcionalidades na gestão de apartamentos são:

- **Criação de apartamentos:** Cria um apartamento com os seguintes atributos: ID do apartamento, nome, endereço, valor da diária, capacidade de hóspedes, disponibilidade e se possui Wi-Fi ou não.
- **Atualização de apartamentos:** Atualiza os atributos do apartamento, caso seja necessário.
- **Remoção de apartamentos:** Remove o apartamento da lista de apartamentos.

Exemplo de código para a atualização de um apartamento:

```
1 public static bool AtualizarApartamento(int id, int idNovo, string nome, string endereco
2     , double valorDiaria, int c, bool disponivel, bool wifi)
3 {
4     Apartamento ap = BuscarApartamentoPorId(id);
5     Apartamento ap2 = BuscarApartamentoPorId(idNovo);
6
7     if (ap == null)
8     {
9         throw new ExcecaoApartamento(2);
10    }
11    if (ap2 != null)
12    {
13        throw new ExcecaoApartamento(1);
14    }
15
16    if (ValidarApartamentos.ValidarApartamento(idNovo, nome, endereco, valorDiaria, c)
17        == false)
18    {
19        return false;
20    }
21
22    ap.Id = idNovo;
23    ap.Nome = nome;
24    ap.Endereco = endereco;
25    ap.ValorDiaria = valorDiaria;
26    ap.Capacidade = c;
27    ap.Disponivel = disponivel;
28    ap.PossuiWifi = wifi;
29
30    return true;
31 }
```

Código 2: Atualizar apartamento

3.3 Gestão de Utilizadores

As principais funcionalidades na gestão de utilizadores são:

- **Criação de utilizadores:** Cria um utilizador com os seguintes atributos: ID do utilizador, permissão, nome e contato.
- **Atualização de utilizadores:** Atualiza os atributos do utilizador, caso seja necessário.
- **Remoção de utilizadores:** Remove um utilizador da lista de utilizadores.

Exemplo de código para a remoção de um utilizador:

```
1 public static bool RemoverUser(int id)
2 {
3     Utilizador u = BuscarUserPorId(id);
4     if (u == null)
5     {
6         throw new ExcecaoUtilizadores(2);
7     }
8     else
9     {
10        ListaUtilizadores.Remove(u);
11        return true;
12    }
13 }
```

Código 3: Remover um utilizador

4 Aspectos Adicionais do Projeto

Além das funcionalidades principais da reserva, apartamentos e utilizadores, o projeto tem outros aspetos importantes para o seu funcionamento. Esses aspetos incluem tratamento de exceções, regras de negócio, validações, entre outros.

4.1 Regras de Negócio

As regras de negócio foram implementadas de forma a garantir que as operações de gestão de reservas, apartamentos e utilizadores respeitem as regras específicas de permissão.

Algumas das regras de negócio são:

- Apenas utilizadores com permissões de administrador ou moderador (3 ou 2) podem realizar operações de atualização e remoção de reservas e apartamentos.
- Reservas podem ser feitas por qualquer utilizador, mas apenas administradores (3) podem excluir uma reserva de forma permanente (Excluí-la da lista).
- Um utilizador não pode remover ou modificar outro utilizador com a mesma ou maior permissão.
- Apenas administradores podem criar apartamentos e utilizadores.

Essas regras foram implementadas nas classes da camada de **RegrasDeNegocio**, garantindo que as permissões sejam respeitadas durante a execução do projeto. Abaixo está um exemplo de uma regra de negócio:

```
1 public static bool TentaCriarUser(int idU, int id, int perm, string nome, string contato
2 )
3 {
4     Utilizador u = Utilizadores.BuscarUserPorId(idU);
5
6     if (u == null)
7     {
8         throw new ExcecaoUtilizadores(2);
9     }
10
11     if (u.Perm == 3)
12     {
13         if (Utilizadores.CriarUser(id, perm, nome, contato) == true)
14         {
15             return true;
16         }
17         return false;
18     }
19     throw new ExcecaoReservaRegras(1);
20 }
```

Código 4: Criar um Utilizador

4.2 Exceções e Tratamento de Erros

O tratamento de exceções foi essencial durante o desenvolvimento do projeto, garantindo que qualquer erro durante a execução seja capturado (catch) e tratado de forma adequada. O projeto conta com uma série de classes, cada uma com vários tipos de exceções específicas para cada tipo de erro.

Algumas das exceções implementadas incluem:

- **ExcecaoApartamentoRegras:** Lançada quando ocorre um erro relacionado a uma regra de negócio, por exemplo tentar adicionar um apartamento sem ser administrador (3).
- **ExcecaoReservaRegras:** Lançada quando corre um erro relacionado a uma regra de negócio, como por exemplo tentar cancelar uma reserva sendo utilizador (1).
- **ExcecaoUtilizadores:** Lançada quando ocorre um erro relacionado a utilizadores, como por exemplo tentar adicionar um utilizador que já existe.

Essas exceções garantem que o projeto funcione corretamente, não permitindo fazer operações inválidas, fornecendo mensagens de erros específicas.

Abaixo está um exemplo de classe de exceções.

```

1 public class ExcecaoUtilizadores : ApplicationException
2 {
3     #region Attributes
4     private int codigoErro;
5
6     #endregion
7
8     private static readonly Dictionary<int, string> UtilizadoresErros = new Dictionary<
9         int, string>()
10    {
11        { 1, "User already exists." },
12        { 2, "User do not exist." },
13        { 3, "Invalid Permission." },
14        { 4, "Cant create user with that ID because one already exists." },
15        { 5, "Invalid ID." },
16        { 6, "Invalid File Name." }
17    };
18
19    #region Constructors
20    public ExcecaoUtilizadores(int error) : base(UtilizadoresErros.ContainsKey(error) ?
21        UtilizadoresErros[error] : "Unknown Error")
22    {
23        CodigoErro = error;
24        Console.WriteLine("\nError: " + CodigoErro + " -> " + Message);
25    }
26
27    #endregion
28
29    #region Properties
30    public int CodigoErro
31    {
32        get { return codigoErro; }
33        set { codigoErro = value; }
34    }
35
36    #endregion
37 }
```

Código 5: Exceções de Utilizador

4.3 Validações de Dados

Além das exceções e regras de negócio, o projeto também inclui uma série de validações, de forma a garantir que os dados sejam corretos ao adicionar ou atualizar em uma lista.

Essas validações são implementadas na biblioteca `Validacoes`, e garantem que dados incorretos ou incompletos não sejam adicionados ou atualizados na respectiva lista. Abaixo estão exemplos de validação de dados:

```
1 public static bool ValidarApartamento(int id, string nome, string endereco, double
   valorDiaria, int c)
2 {
3     if (ValidarID(id) != true)
4     {
5         return false;
6     }
7
8     if (ValidarEndereco(endereco) != true)
9     {
10        return false;
11    }
12
13    if (ValidarValorDiaria(valorDiaria) != true)
14    {
15        return false;
16    }
17
18    if (ValidarCapacidade(c) != true)
19    {
20        return false;
21    }
22
23    return true;
24 }
```

Código 6: Validação Geral

```
1 public static bool ValidarID(int id)
2 {
3     if (id <= 0)
4     {
5         throw new ExcecaoValidarApartamento(1);
6     }
7     return true;
8 }
9
10 public static bool ValidarEndereco(string endereco)
11 {
12     if (endereco == null)
13     {
14         throw new ExcecaoValidarApartamento(2);
15     }
16     return true;
17 }
18
19 public static bool ValidarValorDiaria(double valorDiaria)
20 {
21     if (valorDiaria < 0)
22     {
23         throw new ExcecaoValidarApartamento(3);
24     }
25 }
```

```
25     return true;
26 }
27 public static bool ValidarCapacidade(int c)
28 {
29     if (c <= 0)
30     {
31         throw new ExcecaoValidarApartamento(4);
32     }
33     return true;
34 }
```

Código 7: Validações Específicas

4.4 Interação entre Camadas

O fluxo do projeto respeita um padrão claro, onde cada camada interage apenas com as que necessita. As camadas seguem o seguinte fluxo.

- A camada Main Inicia o programa e interage com a camada RegrasDeNegocio, Excecoes, Dados e ObjetosDeNegocio.
- A camada Dados e ObjetosDeNegocio são uma exceção pois apenas comunica com estes para preparar o programa(criar um utilizador ADMIN).
- A camada RegrasDeNegocio interage com a camada Dados, ObjetosDeNegocio e Excecoes.
- Por fim, a camada Dados interage com ObjetosDeNegocio, Excecoes e Validacoes.

Este fluxo assegura que o projeto funcione de forma eficiente. Respeitando a arquitetura N-Tier, onde cada camada tem a sua responsabilidade.

```
/// \file Utilizadores.cs
/// \brief Classe para gerir utilizadores do sistema.
/// \details Esta classe permite criar, atualizar, remover informações sobre utilizadores.
/// \author Tiago Ferreira
/// \date 2024

using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Runtime.Serialization.Formatters.Binary;
using System.Text;
using System.Threading.Tasks;

using ObjetosDeNegocio;
using Excecoes;
using Validacoes;

namespace Dados
```

Figura 2: Camadas usadas na camada Dados

5 Dificuldades e Soluções

O desenvolvimento do sistema de gestão de alojamentos turísticos apresentou vários desafios, que foram resolvidos ao longo do trabalho. Abaixo estão descritas duas dificuldades significativas e as soluções aplicadas.

Durante o desenvolvimento da fase 2 do projeto, houve alguns desafios, que foram resolvidos ao longo do tempo. Abaixo estão os principais:

5.1 Guardar Listas Em Ficheiros Binários

Dificuldade: Garantir que as listas eram corretamente guardadas em ficheiros binários, sem perder nenhum tipo de dado durante o processo de salvar.

```
1 public static bool GuardarReservasFicheiro(string ficheiro)
2 {
3     //Verificar se a string com nome do ficheiro e nulo ou esta vazio
4     if (ficheiro == "" || ficheiro == null)
5     {
6         throw new ExcecaoReservas(5);
7     }
8     //Criacao do ficheiro e adicionar dados da lista
9     Stream stream = File.Open(ficheiro, FileMode.OpenOrCreate, FileAccess.ReadWrite);
10    BinaryFormatter bin = new BinaryFormatter();
11    bin.Serialize(stream, ListaReservas);
12    stream.Close();
13
14    return true;
15 }
```

Código 8: Guardar Ficheiro Binario

5.2 Ler Listas Em Ficheiros Binários

Dificuldade: Garantir que os ficheiros binários eram corretamente lidos e a informação ser bem guardada numa lista, sem perder nenhum tipo de dado durante o processo de ler.

```
1 public static bool LerReservasFicheiro(string ficheiro)
2 {
3     //Verificar se a string com nome do ficheiro e nulo ou esta vazio
4     if (ficheiro == "" || ficheiro == null)
5     {
6         throw new ExcecaoReservas(5);
7     }
8     //Leitura do ficheiro e adicionar dados a lista
9     Stream stream = File.Open(ficheiro, FileMode.Open);
10    BinaryFormatter bin = new BinaryFormatter();
11    ListaReservas = (List<Reserva>)bin.Deserialize(stream);
12    stream.Close();
13
14    return true;
15 }
```

Código 9: Ler Ficheiro Binario

6 Melhorias Futuras

Embora o trabalho desenvolvido atenda aos requisitos da fase 2, é possível implementar diversas melhorias no futuro, de modo a melhorar o projeto. Algumas das sugestões incluem:

6.1 Persistência de Dados

Atualmente, os dados são armazenados separadamente em ficheiros binários nomeadamente `apartamentos.bin`, `reservas.bin` e `utilizadores.bin`. Para uma melhor organização, seria vantajoso ter todas as listas em um único ficheiro.

6.2 Interface Gráfica

Uma interface gráfica para o projeto (GUI), tornaria-o mais fácil de utilizá-lo. Ferramentas como `Windows Forms` podem ser utilizadas para desenvolver uma aplicação gráfica proporcionando uma interface fácil de utilizar.

6.3 Integração com uma Base de Dados

Em vez de utilizar ficheiros, poderia-se adicionar uma Base de Dados, como `MySQL` ou `SQLite`. Isto permitiria uma gestão mais eficiente dos dados.

7 Diagrama de Classes

O diagrama de classes a seguir representa a estrutura de classes desenvolvidas para o projeto de gestão de alojamentos, incluindo relações de herança.

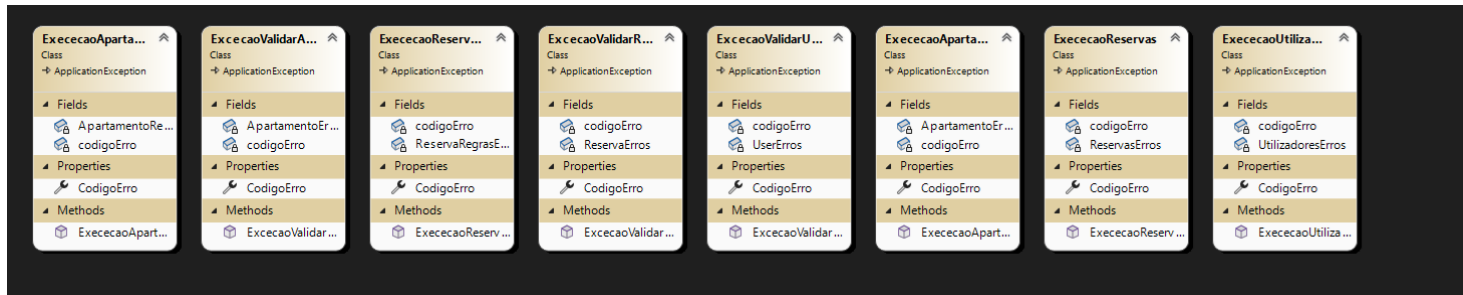


Figura 3: Diagrama de Classes do Sistema (Exceções)

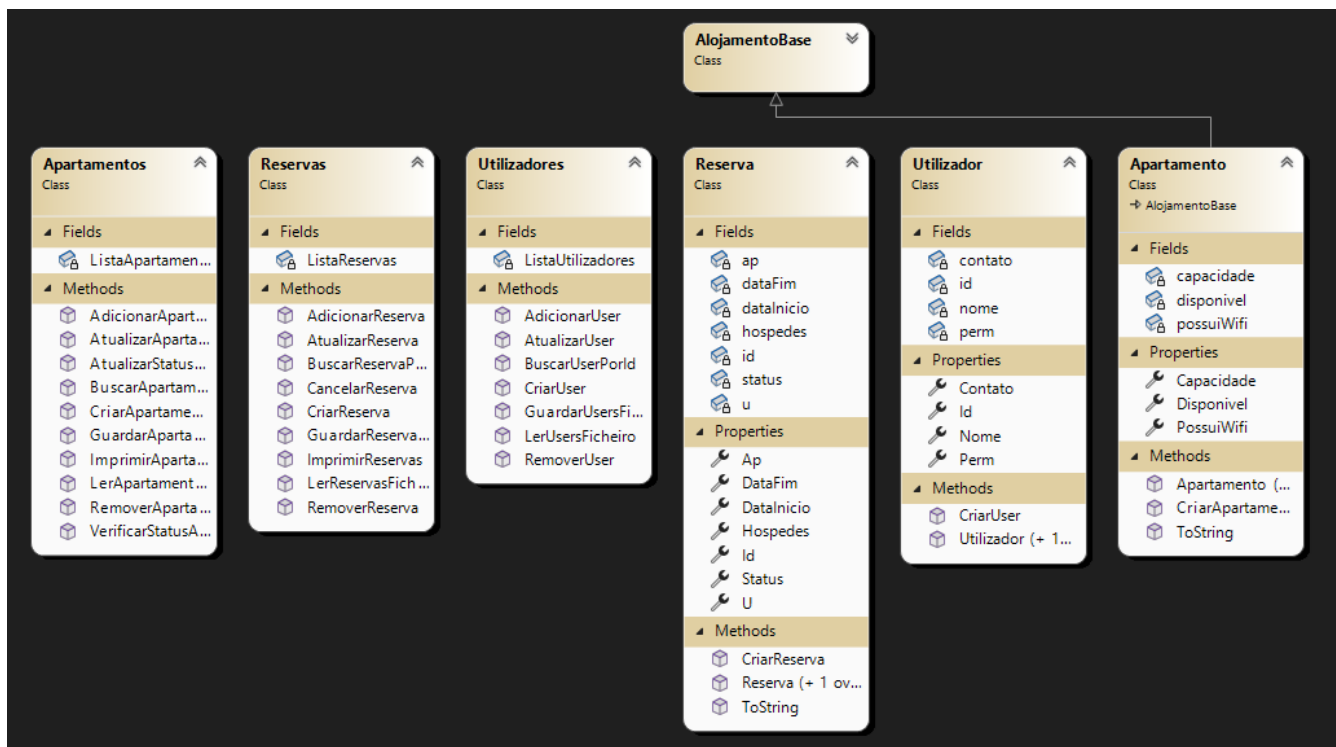


Figura 4: Diagrama de Classes do Sistema (Objetos e Dados)

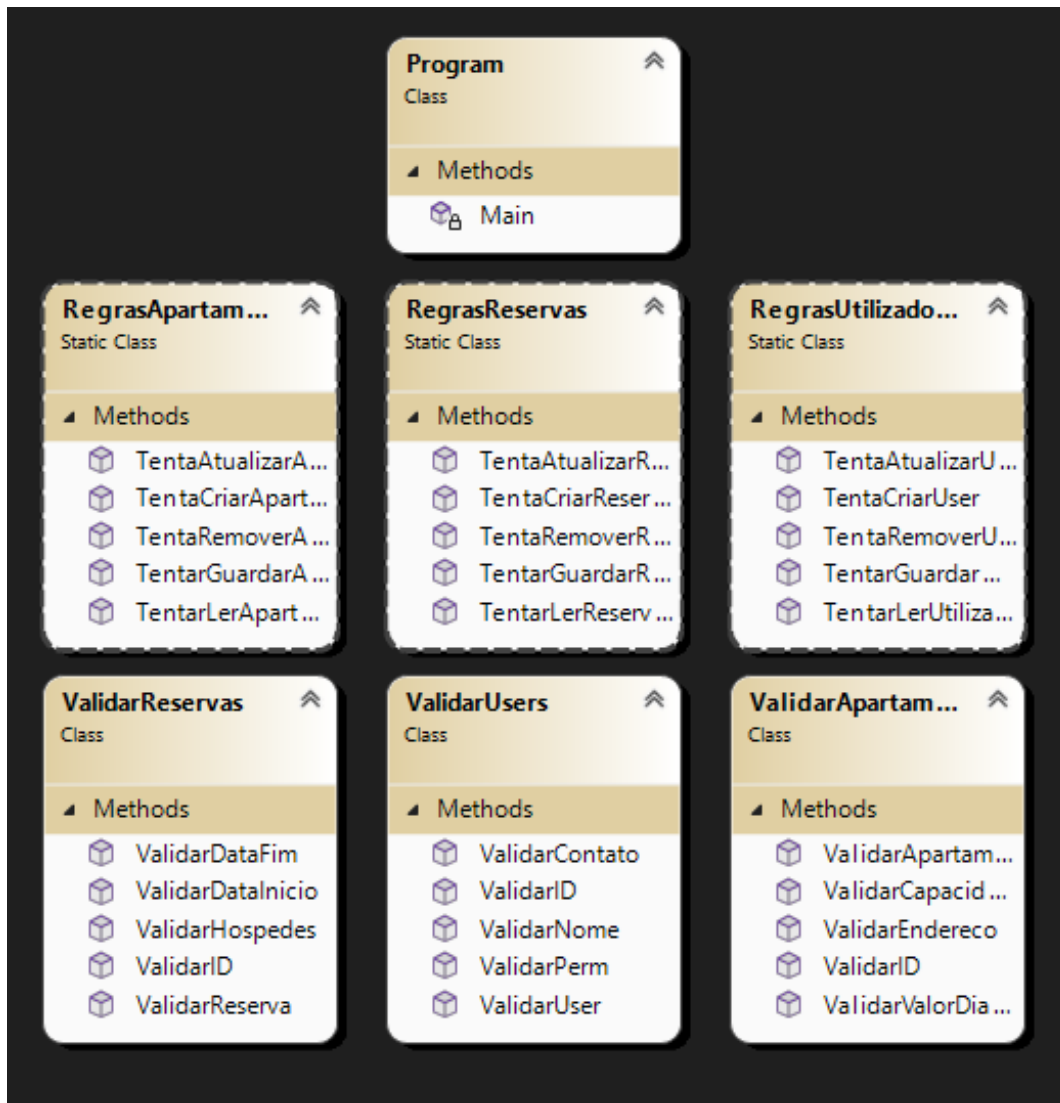


Figura 5: Diagrama de Classes do Sistema (Regras de Negocio, Main e Validações)

8 Conclusão

Este trabalho permitiu explorar e aplicar os conceitos aprendidos na cadeira de Programação Orientada a Objetos (POO), através do desenvolvimento de um projeto de gestão de alojamentos turísticos dividido em várias camadas(cada uma com sua responsabilidade), utilizando a arquitetura *N-Tier*. Utilizar esta determinada arquitetura permitiu uma melhor organização do código o que acaba por facilitar sua manutenção.

O sistema demonstrou ser funcional e eficiente, permitindo:

- Adicionar, listar, atualizar e remover clientes, apartamentos e reservas.
- Gerir a disponibilidade de apartamentos.
- Associar clientes e alojamentos a reservas de forma consistente.

Apesar de fazer todo os objetivos propostos, há espaço para implementar melhorias, como por exemplo integrar uma interface gráfica (GUI), guardar toda informação apenas em um ficheiro e adicionar uma base de dados. Estas melhorias poderiam ainda tornar o projeto mais eficiente.